

# Module 1 – Program Structure & Packages

## Anatomy of a Java Class

- A Java file can contain one public class matching the filename.
- Structure typically follows this sequence:
  1. `package` declaration
  2. `import` statements
  3. `class` definition
  4. `main()` entry method
- The JVM starts execution from the `main()` method.

## Code Snippet

```
package com.learnato.training;

import java.util.List;

public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java 21!");
    }
}
```

## Packages & Sub-Packages

- Packages group related classes into namespaces.
- Sub-packages **do not inherit** visibility from parent packages.
- Naming convention → reverse domain (e.g., `com.learnato.training.util`).
- Use folders to represent packages physically.

## Directory Layout

```
src/  
└─ com/  
    └─ learnato/  
        ├── model/  
        ├── service/  
        └─ util/
```

## Import and Static Import

- `import` allows using classes from other packages without full qualification.
- `import static` lets you use constants or static methods directly.
- Avoid `import *` for cleaner compilation and readability.
- Resolve naming conflicts with fully qualified class names.

### Code Snippet

```
import java.util.Collections;
import static java.lang.Math.*;

public class Demo {
    public static void main(String[] args) {
        System.out.println(max(10, 20)); // from Math
        Collections.emptyList();
    }
}
```

## Access Modifiers & Package Visibility

- `public`: accessible from anywhere.
- `protected`: accessible within same package and subclasses.
- **Default (no modifier)**: package-private.
- `private`: only within the same class.
- Packages help **control encapsulation** and module boundaries.

### Code Snippet

```
package com.learnato.model;

public class Student {
    String name; // package-private
    private int age;
}
```

## Common Mistakes

- Mismatch between package name and folder path → `ClassNotFoundException`.
- Multiple public classes in one file → compile error.
- Missing `main()` → runtime error.
- Incorrect classpath → “Could not find or load main class”.

## Key Takeaways

- Package = namespace = folder hierarchy.
- Imports simplify class usage; static imports simplify constants/methods.
- Access modifiers + packages enforce modular design.
- Understanding compilation & classpath is essential for debugging.
- Strong foundation for modular programming & certification questions.

## Module 2 – Primitive Data Types & Literals

### Overview of Primitive Types

- 8 types: byte, short, int, long, float, double, char, boolean
- Stored by value; fixed size; platform independent
- Ideal for numeric and logical operations

| Type    | Size (bits) | Range                                  | Default Value |
|---------|-------------|----------------------------------------|---------------|
| byte    | 8           | −128 to 127                            | 0             |
| short   | 16          | −32,768 to 32,767                      | 0             |
| int     | 32          | −2 <sup>31</sup> to 2 <sup>31</sup> −1 | 0             |
| long    | 64          | −2 <sup>63</sup> to 2 <sup>63</sup> −1 | 0L            |
| float   | 32          | ≈3.4e38                                | 0.0f          |
| double  | 64          | ≈1.7e308                               | 0.0d          |
| char    | 16          | '\u0000' to '\uffff'                   | '\u0000'      |
| boolean | 1           | true/false                             | false         |

## Default Values & Initialization

- Local variables → must be initialized before use
- Instance variables → auto-initialized (default values)
- `final` variables require explicit initialization

```
public class Defaults {  
    int count;           // 0  
    boolean active;      // false  
    double ratio;        // 0.0  
}
```

## Numeric Literals

### Key Points

- Decimal  $\rightarrow$  `int x = 42;`
- Binary  $\rightarrow$  `0b101010;`
- Hex  $\rightarrow$  `0x2A;`
- Octal  $\rightarrow$  `052;` (legacy)
- Suffixes  $\rightarrow$  L, F, D

```
int binary = 0b1010;  
int hex = 0xFF;  
double num = 123.45d;  
float pi = 3.14f;
```



## Underscore Notation in Literals

- Improves readability: `int million = 1_000_000;`
- Cannot start/end with `_` or appear near decimal/suffix

```
int cc = 1234_5678;  
double balance = 12_345.67; //   
int invalid = _1000; // 
```

## Type Casting & Promotion

- **Widening:** automatic & safe
- **Narrowing:** manual & may lose data
- Promotions occur in expressions

```
byte b = 10;  
int i = b;           // Widening  
byte x = (byte) i;   // Narrowing
```

## Type Promotion in Expressions

- Smaller types (auto) promote to `int` in arithmetic
- Mixed types → promoted to the largest operand type
- Common exam trap → expect `int` when mixing `byte` and `short`

```
byte a = 5, b = 6;  
int c = a + b;    // auto-promoted to int  
double avg = (a + b) / 2.0;
```

## Wrapper Constants

- Wrapper classes define range constants:  
Byte.MIN\_VALUE, Integer.MAX\_VALUE, etc.
- Useful for boundary testing and overflows

```
System.out.println("Max Int: " + Integer.MAX_VALUE);  
System.out.println("Min Int: " + Integer.MIN_VALUE);
```

## Key Takeaways

- 8 primitive types → core of Java's type system
- Fixed sizes ensure portability
- Literals can use binary/hex/underscore notation
- Casting controls data precision and range
- Promotion rules impact expressions and exam questions

# Module 3 – Arithmetic & Logical Operations

## Arithmetic Operators

### Key Points

- Basic arithmetic: +, -, \*, /, %
- Division of integers truncates fractional part.
- Unary ++ and -- increment/decrement by 1.
- Parentheses change natural precedence.

### Code Snippet

```
int a = 7, b = 2;  
System.out.println(a / b); // 3 (integer division)  
System.out.println(a % b); // 1 (remainder)  
a++; // post-increment
```

## Relational Operators

### Key Points

- Compare values → result is boolean.
- >, <, >=, <=, ==, !=
- Works on numbers and characters (Unicode order).

### Code Snippet

```
System.out.println(10 > 5);      // true
System.out.println('A' < 'a');  // true
System.out.println(10 == 10.0); // true  (numeric promotion)
```

## Logical Operators (&&, ||, !)

### Key Points

- Combine or invert boolean results.
- && AND, || OR, ! NOT.
- && and || short-circuit (only evaluate right side if needed).
- Use for conditional flow.

### Code Snippet

```
int x = 5;
if (x < 10 && ++x > 5) {
    System.out.println("x incremented to " + x);
}
```



## Bitwise Operators

### Key Points

- Operate on bits of integer types.
- `&`, `|`, `^`, `~`, `<<`, `>>`, `>>>`.
- Common in flags, masks, cryptography, and performance code.

### Code Snippet

```
int n = 5;           // 0101
System.out.println(n << 1); // 10
System.out.println(~n);  // bitwise NOT
```

## Ternary Operator (?:)

### Key Points

- Shorthand for if-else statements.
- Syntax: `condition ? valueIfTrue : valueIfFalse`.
- Returns a value, not a statement.

### Code Snippet

```
int age = 20;  
String category = age >= 18 ? "Adult" : "Minor";  
System.out.println(category);
```

## Operator Precedence & Associativity

### Visual Chart

| Precedence (High → Low) | Example Operators | Associativity |
|-------------------------|-------------------|---------------|
| Unary                   | ++ -- ! ~ + -     | Right → Left  |
| Multiplicative          | * / %             | Left → Right  |
| Additive                | + -               | Left → Right  |
| Relational              | < > <= >=         | Left → Right  |
| Equality                | == !=             | Left → Right  |
| Logical AND/OR          | &&                | Left → Right  |
| Ternary                 | ?:                | Right → Left  |
| Assignment              | = += -=           | Right → Left  |

## Type Promotion in Arithmetic

### Key Points

- Smaller than int ( byte, short, char ) → promoted to int.
- Mixed types → promoted to largest operand type.
- Causes unexpected results with overflow.

### Code Snippet

```
byte a = 5, b = 6;  
int c = a + b;           // promoted to int  
double avg = (a + b) / 2.0; // floating division
```

## Compound Assignments

### Key Points

- Shorthand for operation + assignment: +=, -=, \*=, /=, %=.
- Implicitly performs type cast.

```
byte b = 10;  
b += 5;    // OK (auto-cast)  
b = b + 5; // ❌ compile error
```

## Common Traps & Exam Pitfalls

### Key Points

- Integer division vs floating point.
- `==` vs `.equals()` in boolean logic.
- Mis-grouped expressions due to precedence.
- Overflow in intermediate calculations.

## **Key Takeaways**

- Memorize operator precedence (order matters).
- Distinguish integer vs floating division.
- Trace boolean evaluation for short-circuiting.
- Expect type promotion and overflow questions in the exam.
- Reinforce concepts through print-tracing and debugging.

## Module 4 – `var` Keyword & Type Inference

### What Is `var`?

#### Key Points

- Added in Java 10 for local variable type inference.
- Lets the compiler deduce the type from the initializer.
- Improves readability for long generic types.
- Type is still *static* and checked at compile time.

#### Code Snippet

```
var list = new ArrayList<String>();  
list.add("Learnato");  
String value = list.get(0);
```



## Rules & Restrictions

### Key Points

- Can be used **only for local variables** (not fields, parameters, return types).
- Must have an initializer so the compiler can infer type.
- Cannot initialize with `null`.
- Not valid for lambda parameters without parentheses.

### Code Snippet

```
var x = 10;           // int
var name = "Java";   // String
var n = null;         // ❌ Compile error
```

## Type Inference in Loops and Try-with-Resources

### Key Points

- Works inside `for` loops and enhanced loops.
- Useful in `try-with-resources` when type is clear.

### Code Snippet

```
for (var i = 0; i < 5; i++) {  
    System.out.println(i);  
}  
  
try (var reader = new BufferedReader(new FileReader("data.txt"))) {  
    System.out.println(reader.readLine());  
}
```

## Type Inference with Generics

### Key Points

- The inference comes from the right-hand side.
- Always provide type parameters on object creation for clarity.
- Ambiguous inference can lead to `Object` type.

### Code Snippet

```
var map = new HashMap<String, Integer>(); // Inferred as HashMap<String, Integer>
var list = List.of(1, 2, 3);             // Inferred as List<Integer>
```

## Limitations and Pitfalls

### Key Points

- Overuse can hurt readability.
- Avoid `var` when initializer type is not obvious.
- No support for array declarations without initializer.
- Avoid mixing primitive and boxed types unexpectedly.

### Code Snippet

```
var data = process(); // ✗ unclear return type  
var value = 3.14;     // inferred double, not float
```

## **var vs Explicit Typing**

### **Key Points**

- Use `var` for clarity, not laziness.
- Explicit types are better for API and team code.
- Combine both styles strategically.

### **Code Snippet**

```
// Readable
var users = new ArrayList<User>();

// Prefer explicit in method signature
List<User> getActiveUsers() { ... }
```

## Exam Traps

### Key Points

- `var` does not change runtime type.
- Cannot declare multiple `vars` in one statement.
- Inference is compile-time — no polymorphism involved.

### Code Snippet

```
var x = 1, y = 2; // ❌ invalid
```

## Key Takeaways

- `var` infers type from initializer at compile time.
- Cannot be used for fields or method signatures.
- Must be initialized and cannot be `null`.
- Improves readability when used judiciously.
- Common trap: “`var` is like Python’s dynamic typing” → **✗** false.

# Module 5 – Wrapper Classes & Autoboxing

## Purpose of Wrapper Classes

### Key Points

- Wrappers encapsulate primitives into objects.
- Required for:
  - Collections (`List<Integer>` not `List<int>`).
  - Generics and reflection APIs.
  - Null handling and immutability.
- Found in `java.lang`: `Integer`, `Double`, `Boolean`, etc.

### Code Snippet

```
int n = 10;
Integer obj = Integer.valueOf(n); // Manual boxing
int val = obj.intValue();         // Manual unboxing
```



## Autoboxing and Unboxing

### Key Points

- Introduced in Java 5 — automatic conversion between primitive and wrapper.
- Reduces boilerplate but can hide performance cost.
- Happens during assignments and expressions.

### Code Snippet

```
Integer a = 10;    // autoboxing
int b = a;         // unboxing
System.out.println(a + 5); // auto-unbox then add
```

## Wrapper Comparison: == vs equals()

### Key Points

- == compares object references.
- .equals() compares values.
- Integer cache (−128 to 127) causes confusing results.
- Always use .equals() for value comparison.

### Code Snippet

```
Integer x = 127, y = 127;  
System.out.println(x == y); // true (cached)  
Integer p = 128, q = 128;  
System.out.println(p == q); // false
```

## Wrapper Utilities and Constants

### Key Points

- Each wrapper provides utility methods:
  - `parseXxx(String)` → **primitive**
  - `valueOf(String)` → **wrapper**
  - `toString(value)` → **string**
- Useful constants: `MAX_VALUE`, `MIN_VALUE`.

### Code Snippet

```
int num = Integer.parseInt("123");  
System.out.println(Double.MAX_VALUE);
```

## Boxing in Collections and Streams

### Key Points

- Collections and Streams require objects, not primitives.
- Autoboxing occurs automatically inside collections and lambda expressions.

### Code Snippet

```
List<Integer> nums = List.of(1,2,3);  
int sum = nums.stream().mapToInt(Integer::intValue).sum();
```

## Null Handling and Unboxing Risks

### Key Points

- Unboxing null wrapper → `NullPointerException`.
- Always check null before unboxing.
- Prefer `Optional` or default values.

### Code Snippet

```
Integer age = null;  
int safeAge = (age != null) ? age : 0;
```

## **Performance and Memory Impact**

### **Key Points**

- Wrappers consume more memory than primitives.
- Frequent autoboxing in loops → GC pressure.
- Prefer primitive streams and arrays in critical code.

## Wrapper Conversion Matrix

| From      | To        | Method                           | Example                             |
|-----------|-----------|----------------------------------|-------------------------------------|
| String    | Primitive | <code>parseXxx()</code>          | <code>Integer.parseInt("42")</code> |
| String    | Wrapper   | <code>valueOf()</code>           | <code>Double.valueOf("4.2")</code>  |
| Primitive | Wrapper   | <code>Autobox / valueOf()</code> | <code>Integer.valueOf(10)</code>    |
| Wrapper   | Primitive | <code>Unbox / xxxValue()</code>  | <code>obj.intValue()</code>         |

## Common Exam Traps

### Key Points

- Assuming `Integer x = null; int y = x;` is safe.
- Using `==` for wrapper comparison.
- Forgetting caching range (−128 to 127).
- Ignoring autoboxing inside ternary operator.

### Code Snippet

```
Integer a = null;  
System.out.println(a == 0); // NPE
```



## Key Takeaways

- Wrappers enable objects for primitives.
- Autoboxing is convenient but can impact performance.
- Always check null before unboxing.
- Prefer `.equals()` for value comparison.
- Understand Integer caching and exam pitfalls.

# Module 6 – Boolean Expressions & Short-Circuit Evaluation

## Boolean Basics

### Key Points

- Boolean type → `true` or `false` only.
- Operands in conditional statements must be boolean.
- Use `&&`, `||`, `!` for logical operations.
- `&`, `|`, `^` can be bitwise or boolean (non-short-circuit).

### Code Snippet

```
boolean a = true, b = false;  
System.out.println(a && b); // false  
System.out.println(!a);    // false  
System.out.println(a || b); // true
```

## Short-Circuit Operators (&&, ||)

### Key Points

- && — AND → right side evaluated **only if** left is true.
- || — OR → right side evaluated **only if** left is false.
- Prevents unnecessary (or unsafe) evaluation.

### Code Snippet

```
boolean check() {  
    System.out.println("Check called");  
    return true;  
}  
boolean result = false && check(); // "Check called" never prints
```

## Bitwise vs Logical Operators

### Key Points

- `&` and `|` evaluate **both** sides (always).
- `&&` and `||` skip right operand when possible.
- Use bitwise only if both sides must run (e.g., counter updates).

### Code Snippet

```
int x = 0;
if (x < 0 & ++x > 0) { } // both sides run
if (x < 0 && ++x > 0) { } // short-circuited
System.out.println(x);
```

## Order of Evaluation

### Key Points

- Java evaluates from left to right.
- Short-circuiting skips remaining parts once result is known.
- Use this to guard against `NullPointerException`.

### Code Snippet

```
String s = null;
if (s != null && s.length() > 3) {
    System.out.println("Safe check");
}
```

## Combining Conditions

### Key Points

- Combine multiple boolean expressions for complex checks.
- Always group using parentheses for clarity.
- Ternary operator `?:` can work with boolean expressions.

### Code Snippet

```
int age = 20;  
String status = (age >= 18 && age < 60) ? "Adult" : "Not Adult";
```

## Negation and Truth Tables

### Key Points

- $!$  inverts boolean value.
- Truth tables help understand compound logic.

**A B A && B A || B !A**

T T T      T      F

T F F      T      F

F T F      T      T

F F F      F      T

## Precedence and Ternary Mix

### Key Points

- `!` has higher precedence than `&&` or `||`.
- Ternary `?:` is lower than logical operators.
- Always use parentheses for clarity.

### Code Snippet

```
boolean result = true || false && false; // true
boolean res2 = !(true || false) && true; // false
```



## Common Pitfalls

### Key Points

- Using & instead of && can cause unexpected method calls.
- Null checks must always precede method invocations.
- Assignments inside conditions (= vs ==).

### Code Snippet

```
boolean flag = false;  
if (flag = true) System.out.println("Oops!"); // always true
```

## Key Takeaways

- Short-circuiting saves time and prevents errors.
- Always use `&&` and `||` for conditional logic.
- Guard null checks on left side first.
- Trace boolean expressions to understand flow.
- Parentheses = clarity = fewer bugs.

# Module 7 – Math API & BigDecimal Precision Handling

## Math Class Overview

### Key Points

- Located in `java.lang`; contains only static methods and constants.
- Provides mathematical functions and constants (`PI`, `E`).
- Common methods: `abs`, `max`, `min`, `ceil`, `floor`, `sqrt`, `pow`, `round`, `random`.

### Code Snippet

```
System.out.println(Math.abs(-7));  
System.out.println(Math.pow(2, 3));  
System.out.println(Math.sqrt(16));  
System.out.println(Math.random()); // 0.0 ≤ x < 1.0
```

## Rounding and Truncation

### Key Points

- Floating-point numbers cannot always represent decimals exactly.
- Rounding functions help control precision.
- `ceil` → round up, `floor` → round down, `round` → nearest int (half-up).

### Code Snippet

```
System.out.println(Math.ceil(4.3));    // 5.0
System.out.println(Math.floor(4.7));   // 4.0
System.out.println(Math.round(4.5));   // 5
```

## Math Constants and Random Numbers

### Key Points

- `Math.PI`, `Math.E` are commonly used constants.
- `Math.random()` returns pseudo-random double ( $0 \leq x < 1$ ).
- For reproducible randomness → use `java.util.Random` or `SecureRandom`.

### Code Snippet

```
System.out.println(Math.PI);  
System.out.println(Math.random() * 10); // 0-9 range
```

## Why We Need BigDecimal

### Key Points

- `float` and `double` use binary approximation → rounding errors.
- `BigDecimal` provides precise decimal representation.
- Ideal for financial and scientific calculations.

### Code Snippet

```
System.out.println(0.1 + 0.2);           // 0.30000000000000004
BigDecimal a = new BigDecimal("0.1");
BigDecimal b = new BigDecimal("0.2");
System.out.println(a.add(b));           // 0.3
```

## Creating BigDecimal Correctly

### Key Points

- Use string constructor for precise values.
- Avoid `new BigDecimal(double)` — inherits binary imprecision.
- Use `BigDecimal.valueOf(double)` only for known constants.

### Code Snippet

```
BigDecimal bad = new BigDecimal(0.1);
BigDecimal good = new BigDecimal("0.1");
System.out.println(bad); // 0.10000000000000000555...
System.out.println(good); // 0.1
```

## BigDecimal Arithmetic

### Key Points

- Methods: add, subtract, multiply, divide.
- Division may throw `ArithmeticException` if non-terminating decimal → specify scale and rounding mode.
- Immutable objects — each operation returns new instance.

### Code Snippet

```
BigDecimal x = new BigDecimal("10");  
BigDecimal y = new BigDecimal("3");  
System.out.println(x.divide(y, 2, RoundingMode.HALF_UP)); // 3.33
```



# Rounding Modes in BigDecimal

## Key Points

- Common modes:
  - HALF\_UP: round to nearest neighbor (ties up).
  - HALF\_EVEN: Banker's rounding.
  - CEILING / FLOOR: toward  $\pm\infty$ .
  - DOWN: truncate fraction.
- Use `setScale()` to apply mode.

## Code Snippet

```
BigDecimal value = new BigDecimal("2.675");  
System.out.println(value.setScale(2, RoundingMode.HALF_UP)); // 2.68  
System.out.println(value.setScale(2, RoundingMode.HALF_EVEN)); // 2.68
```

## Precision and Scale

### Key Points

- **Precision:** Total significant digits.
- **Scale:** Digits after decimal.
- Use `MathContext` for overall precision control.

### Code Snippet

```
BigDecimal n = new BigDecimal("123.4567");  
System.out.println("Precision: " + n.precision()); // 7  
System.out.println("Scale: " + n.scale());          // 4
```

# BigDecimal in Financial Applications

## Key Points

- Always use `BigDecimal` for currency, tax, or interest rates.
- Avoid `float`/`double` in financial systems.
- Consider `Currency` and `NumberFormat` for localized display.

## Code Snippet

```
BigDecimal price = new BigDecimal("19.99");
BigDecimal tax = new BigDecimal("0.18");
BigDecimal total = price.add(price.multiply(tax)).setScale(2,
RoundingMode.HALF_UP);
System.out.println("Total: ₹" + total);
```

## Performance and Best Practices

### Key Points

- `BigDecimal` is slower but accurate — use only when needed.
- Use primitive `double` for graphics/scientific simulation where tiny error is acceptable.
- Prefer constant scale and context in repetitive operations.

### Key Takeaways

- Use `Math` for quick operations, `BigDecimal` for precision.
- Always specify `RoundingMode` when dividing.
- Avoid `new BigDecimal(double)` — use `String` constructor.
- Understand precision vs scale for exam and real world.

# Module 8 – Common Arithmetic Pitfalls: Division | Overflow | NaN | Infinity

## Integer Division

### Key Points

- Division between integers truncates the fractional part.
- Always yields an integer result (no rounding).
- To get decimals → promote at least one operand to double.

### Code Snippet

```
int a = 7, b = 2;  
System.out.println(a / b);      // 3  
System.out.println((double)a / b); // 3.5
```

## Division by Zero

### Key Points

- **Integer division by 0** → **ArithmeticException**.
- **Floating-point division by 0** → **Infinity** (IEEE 754 standard).
- **0.0 / 0.0** → **NaN** (“Not a Number”).

### Code Snippet

```
System.out.println(5 / 0);           // throws ArithmeticException
System.out.println(5.0 / 0);         // Infinity
System.out.println(0.0 / 0.0);       // NaN
```

## Understanding Infinity

### Key Points

- Positive/Negative Infinity are valid double values.
- Operations propagate Infinity ( $\text{Infinity} + 1 = \text{Infinity}$ ).
- Compare using `Double.isInfinite(value)`.

### Code Snippet

```
double inf = 10.0 / 0;  
System.out.println(inf + 1);           // Infinity  
System.out.println(Double.isInfinite(inf)); // true
```

## NaN ( Not a Number )

### Key Points

- NaN represents an undefined result (e.g.,  $0/0$  or  $\sqrt{-1}$ ).
- $\text{NaN} \neq \text{NaN} \rightarrow$  Always false.
- Use `Double.isNaN(x)` to check.

### Code Snippet

```
double nan = 0.0 / 0.0;  
System.out.println(nan == nan);           // false  
System.out.println(Double.isNaN(nan));    // true
```



## Overflow and Underflow

### Key Points

- When an arithmetic operation exceeds primitive range → value wraps around silently.
- Example: `Integer.MAX_VALUE + 1 = Integer.MIN_VALUE`.
- No exception is thrown!

### Code Snippet

```
int max = Integer.MAX_VALUE;  
System.out.println(max + 1); // -2147483648  
System.out.println(Integer.MIN_VALUE - 1); // 2147483647
```

## Detecting and Preventing Overflow

### Key Points

- Java 8 + methods:
  - `Math.addExact()`, `subtractExact()`, `multiplyExact()` → throw `ArithmeticException` on overflow.
- Use `BigInteger` for arbitrary precision.

### Code Snippet

```
try {  
    int result = Math.addExact(Integer.MAX_VALUE, 1);  
} catch (ArithmeticException e) {  
    System.out.println("Overflow detected!");  
}
```

## Floating-Point Precision Loss

### Key Points

- `double`  $\approx$  15 decimal digits precision  $\rightarrow$  rounding errors possible.
- Repeated addition or subtraction magnifies error.
- Always use `BigDecimal` for currency/financial values.

### Code Snippet

```
double sum = 0.1 + 0.2 + 0.3;  
System.out.println(sum); // 0.6000000000000001
```

## Mixed-Type Arithmetic and Casting

### Key Points

- Mixing int/long/double → promotion to largest type.
- Accidental loss of fractional data if casting back.

### Code Snippet

```
int a = 5;  
double b = 2.0;  
int result = (int)(a / b); // 2 → fraction lost
```

## Exam and Coding Traps

### Common Pitfalls

- $10/0 \rightarrow$  Runtime Exception, not compile error.
- $0.0/0.0 \rightarrow$  NaN (not Infinity).
- Overflow wraps without warning.
- $\text{NaN} \neq \text{NaN}$ ,  $\text{Infinity} == \text{Infinity}$ .
- Use `Math.addExact` or `BigDecimal` for safety.

### Key Takeaways

- Integer division truncates; watch for 0-division.
- Floating-point uses IEEE 754 rules ( $\pm$ Infinity, NaN).
- Overflow is silent — use `addExact()` for detection.
- For money  $\rightarrow$  `BigDecimal`, not `double`.
- Always validate inputs to avoid undefined results.